ROS

colcon, workspace, publisher, subscriber

Branimir Ćaran

8.4.2025.| Regionalni centar izvrsnosti za robotske tehnologije

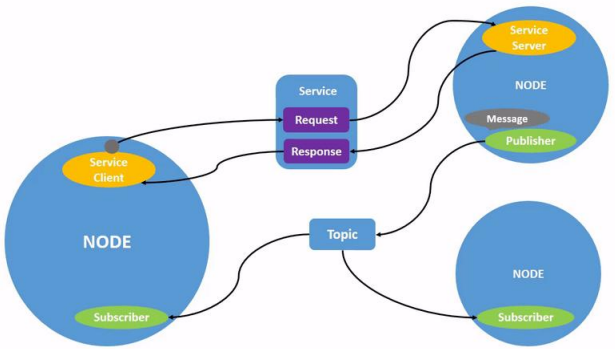
Plan predavanja

- Podsjetnik
- *colcon*
- Izrada *workspace*-a
- Izrada paketa
- Pisanje vlastitog *publisher*-a
- Pisanje vlastitog *subscriber*-a

crta

ROS Node

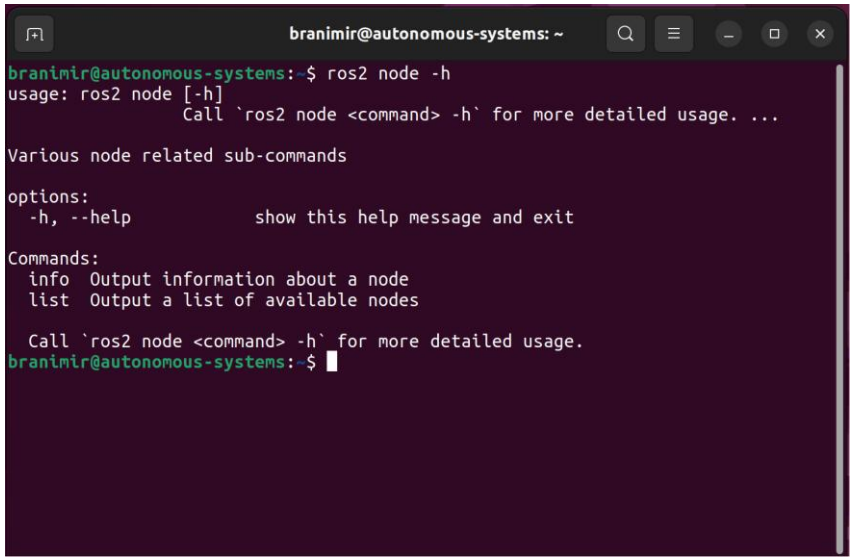
- Čvorovi, odnosno zasebni „programi” koji međusobno komuniciraju
- Komuniciraju na više načina:
  - Preko *topic*-a
  - Preko *service*-a
  - Preko *action*-a




```
ros2 node <command>
```

- Ključne naredbe:
  - *info* – informacije o čvoru
  - *list* – ispis dostupnih/upaljenih čvorova
- Za popis svih naredbi i njihov opis koristiti

```
ros2 node -h
```



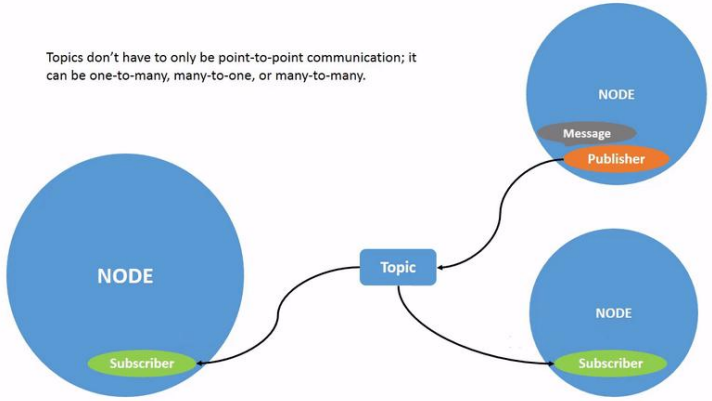
ROS Topic

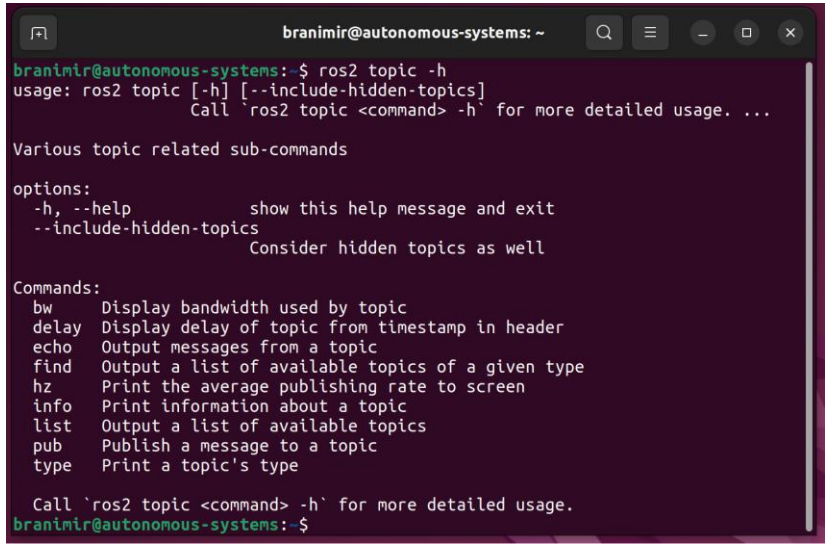
- Kontinuirani protok podataka (*engl. stream*)

```
ros2 topic <command>
```

- Ključne naredbe:
  - *list* – ispis svih dostupnih tema na koje se objavljuje
  - *echo* – osluškivanje podataka koje se objavljuju na određenu temu
  - *pub* – objavljivanje na određenu temu
  - *hz; info; type* itd.
- Za popis svih naredbi i njihov opis koristiti

```
ros2 topic -h
```



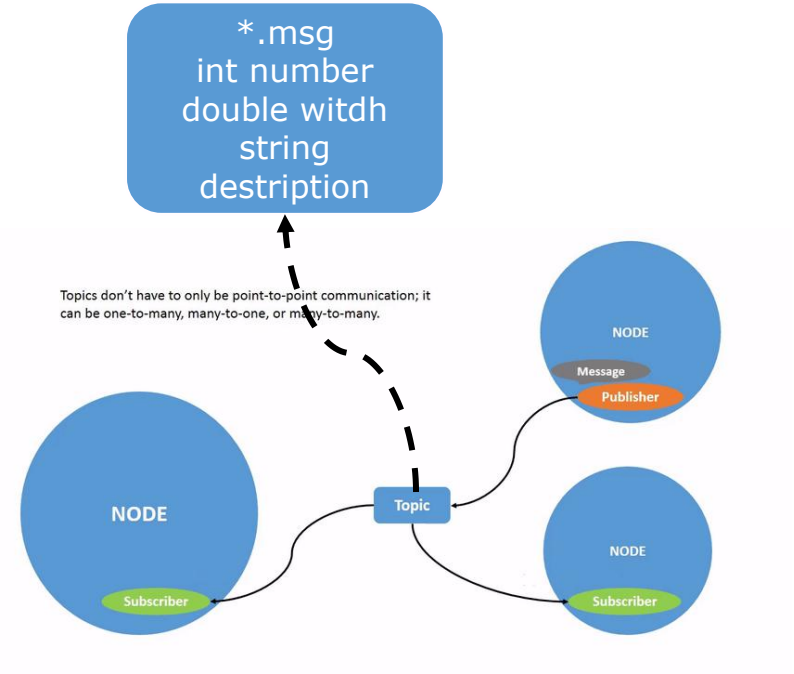


ROS Messages

- Struktura podataka koja definira vrstu *topic*-a
- Sastoji se od ugniježdenih struktura:
  - *Integres, floats, booleans, strings* itd.
- Definirani su u **.msg* datotekama

- Provjera vrste *topic*-a

```
ros2 topic type /ime_topica
```



ROS Messages

geometry_msgs/msg/Point.msg

```
float 64 x
float 64 y
float 64 z
```

geometry_msgs/msg/Pose

```
geometry_msgs/Point position
        float64 x
        float64 y
        float64 z
geometry_msgs/Quaternion orientation
        float64 x
        float64 y
        float64 z
        float64 w
```

sensor_msgs/msg/LaserScan

```
std_msgs/msg/Header
header
float angle_min
float angle_max
float angle_increment
float time_increment
float scan_time
float range_min
float range_max
float[] ranges
float[] intensities
```

std_msgs/msg/Header

```
builtin_interfaces/msg/Time stamp
string frame_id
```

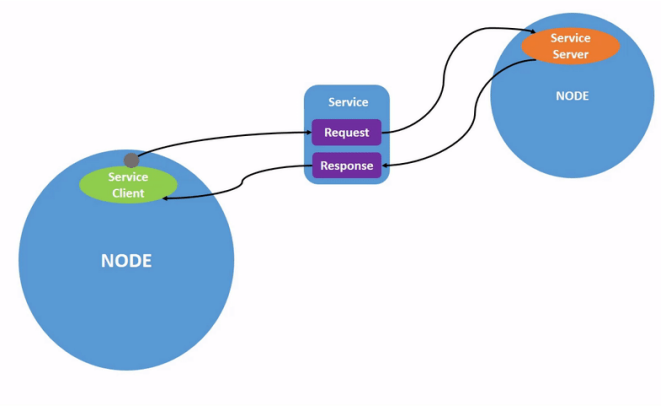
ROS Service

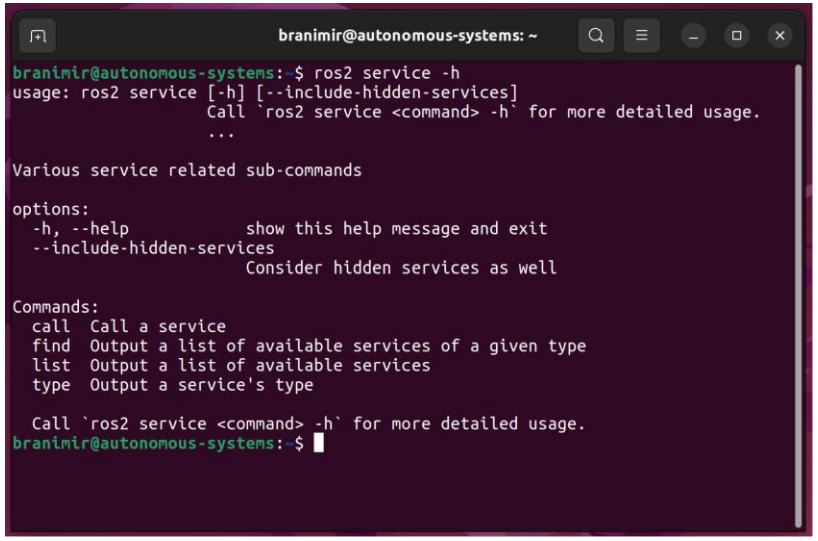
- Obavljanje kraćeg zadatka na zahtjev

```
ros2 service <command>
```

- Ključne naredbe:
  - *call* – poziv/pokretanje servisa
  - *find* – ispis servisa određenog tipa
  - *list* – popis dostupnih servisa
  - *type* – tip servisa
- Za popis svih naredbi i njihov opis koristiti

```
ros2 service -h
```





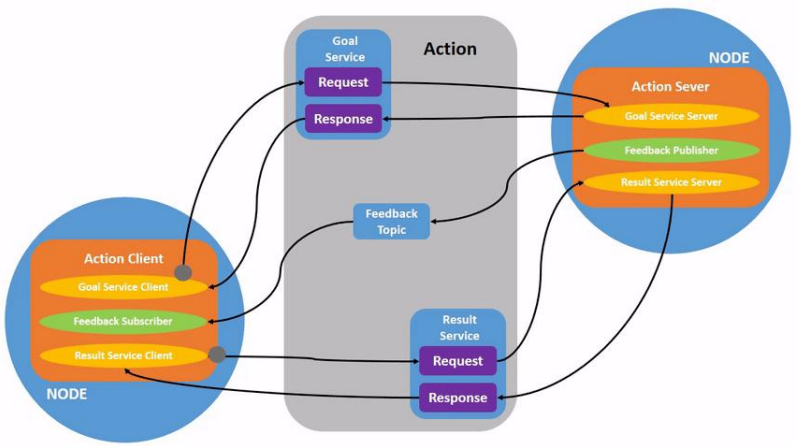
ROS Actions

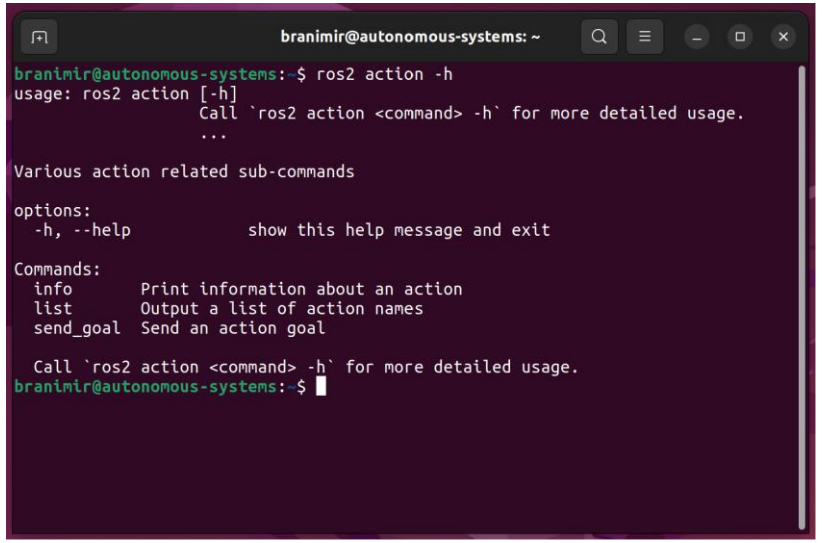
- Parametri koje čvorovi koriste za svoj rad

```
ros2 action <command>
```

- Ključne naredbe:
  - *info* – informacije o akciji
  - *list* – popis dostupnih akcija
  - *send_goal* – slanje cilja akciji
- Za popis svih naredbi i njihov opis koristiti

```
ros2 action -h
```





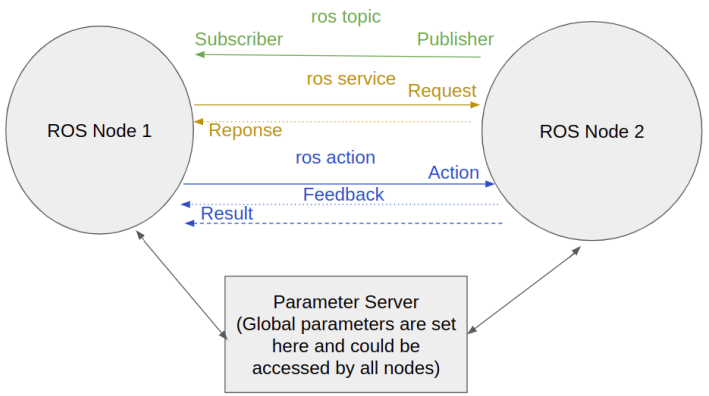
ROS Parameters

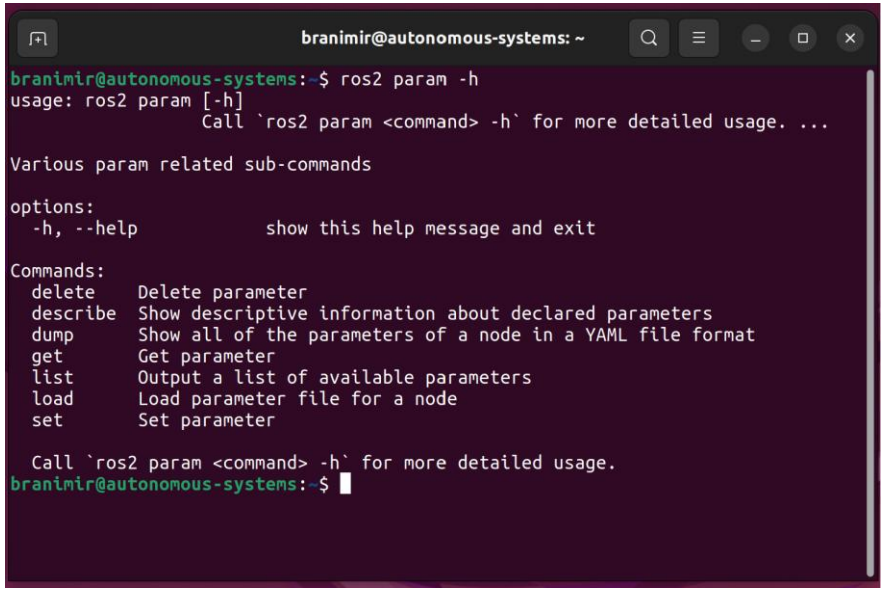
- Kombinacija *topic*-a i *service*-a

```
ros2 param <command>
```

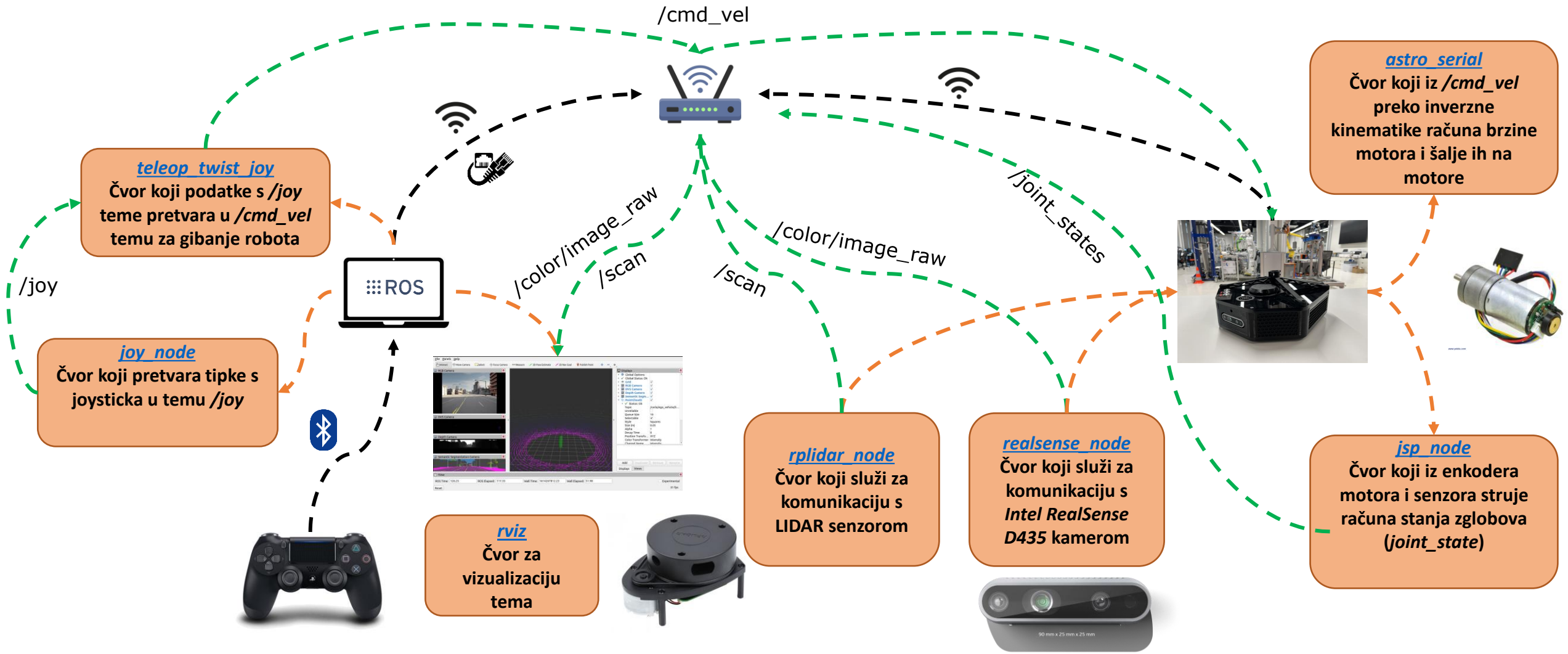
- Ključne naredbe:
  - *get* – dobiti vrijednost parametra
  - *set* – podesiti vrijednost parametra
  - *load* – učitati parametre
  - *dump* – ispisati sve parametre u obliku *YAML* datoteke
  - *delete; describe; list*
- Za popis svih naredbi i njihov opis koristiti

```
ros2 param -h
```

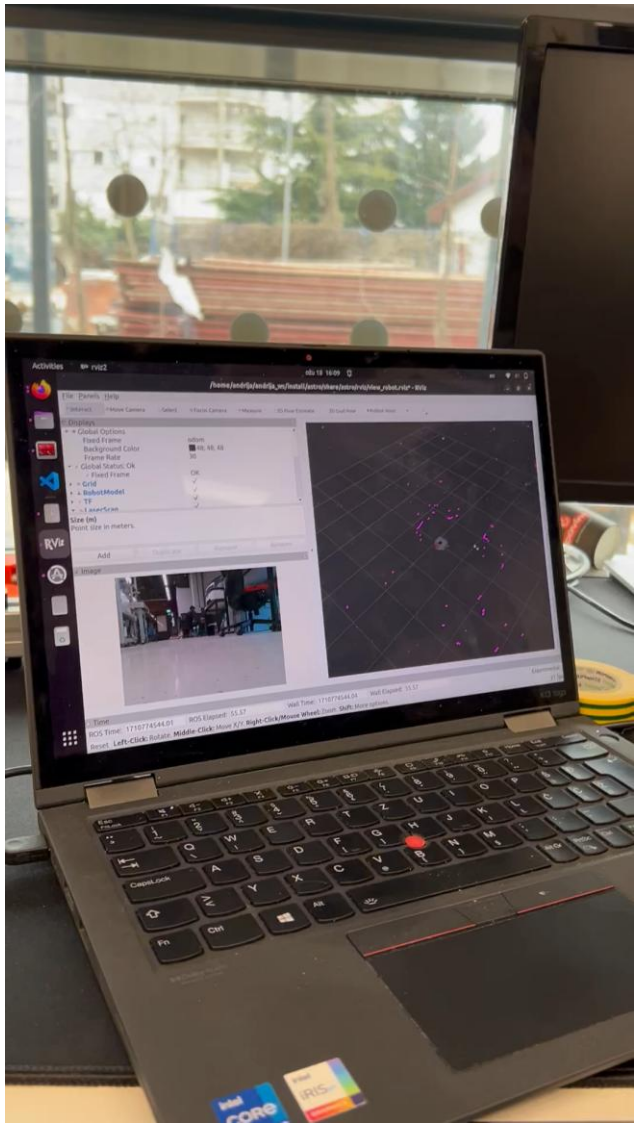




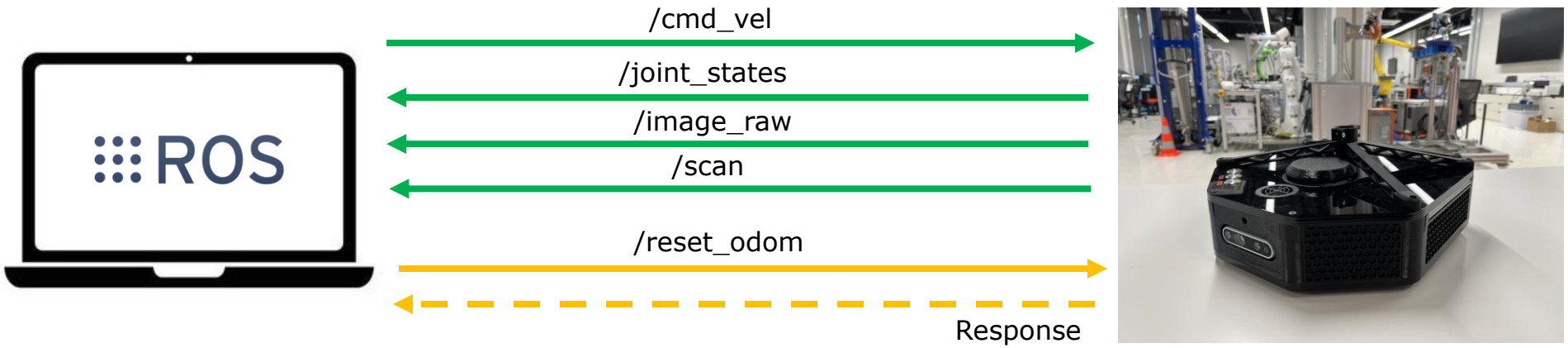
Primjer ASTRO robot



Primjer ASTRO robot



Primjer ASTRO robot



colcon

- *colcon* je iteracija ROS alata za izradu programa poput *catkin_make*, *catkin_make_isolated*, *catkin_tools* i *amnet_tools*
- Potrebno je instalirati *colcon*

```
sudo apt install python3-colcon-common-extensions
```

Osnove

- ROS radni prostor (*engl. workspace*) je direktorij (mapa) s određenom strukturom
- Glavni pod-direktorij je *src* u kojem se nalaze izvorni programi ROS paketa
- Prilikom stvaranja radnog prostora, potrebno je napravi prazni direktorij i pod-direktorij *src*
- *colcon* će prilikom izrade programa stvoriti dodatne direktorije:
  - *build* – direktorij u kojem će se spremati među datoteke, za svaki paket će se napraviti pod-direktorij s npr. *CMake* datoteka za pozivanje
  - *install* – direktorij u koji će se instalirati svaki paket, svaki paket će biti instaliran u zaseban pod-direktorij
  - *log* – direktorij u kojem se nalaze logirane informacije prilikom pozivanja *colcon*-a
- Za razliku od ROS1, ne postoji direktorij *devel*

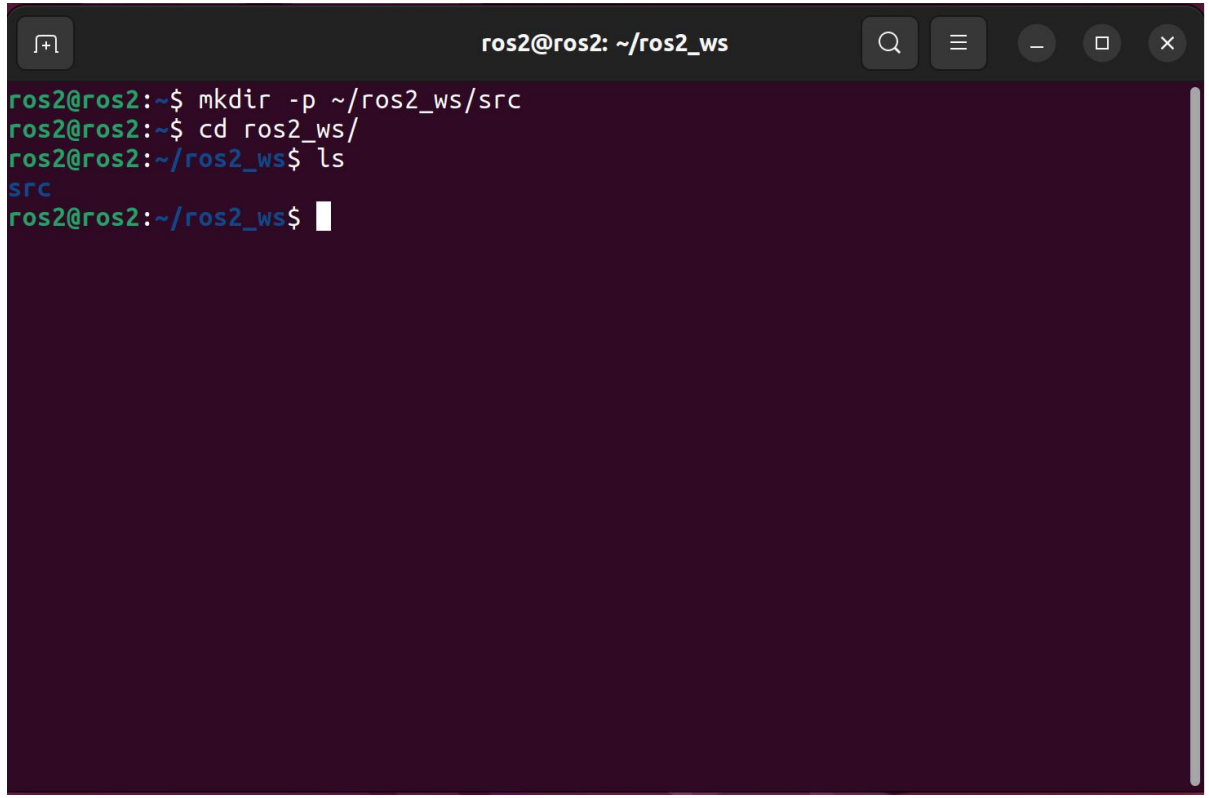
Stvaranje direktorija

- Potrebno je napraviti direktorij radnog prostora *ros2_ws* i pod-direktorij *src*
  - Savjet: koristiti *_ws* kako bi se lakše razumjelo da je riječ o radnom prostoru

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws
```

```
ls
```



Stvaranje direktorija

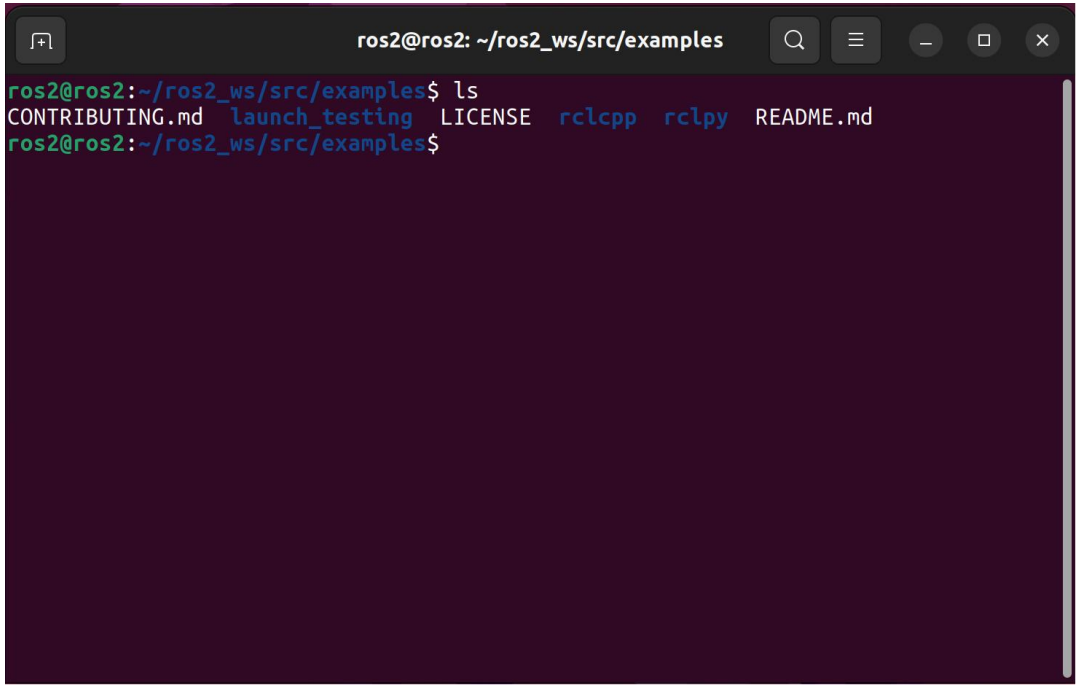
- Potrebno je klonirati primjer s *github*-a u pod-direktorij *src* u pod-direktorij *examples*

```
git clone https://github.com/ros2/examples src/examples -b humble
```

- U preuzetim primjerima se nalaze sljedeći primjeri

```
cd src/examples
```

```
ls
```



- **VAŽNO:**

Naš radni direktorij je „nadređeni” direktorij koji se nalazi iznad glavnog *opt/ros/humble/share* direktorija kojeg smo *source*-ali

Source-anje novog direktorija je važno, ali treba obratiti pažnju ako postoji više vlastitih radnih direktorija

Stvaranje direktorija

ROS
/opt/ros/humble/setup.bash

KUKA robot
kuka_iiwa14_ws

iiwa14_ws/intstall/setup.bash

JACKAL robot
jackal _ws

jackal _ws /intstall/setup.bash

SPOT robot
spot _ws

spot_ws /intstall/setup.bash

Hexsoon 450 dron
drone _ws

drone _ws /intstall/setup.bash

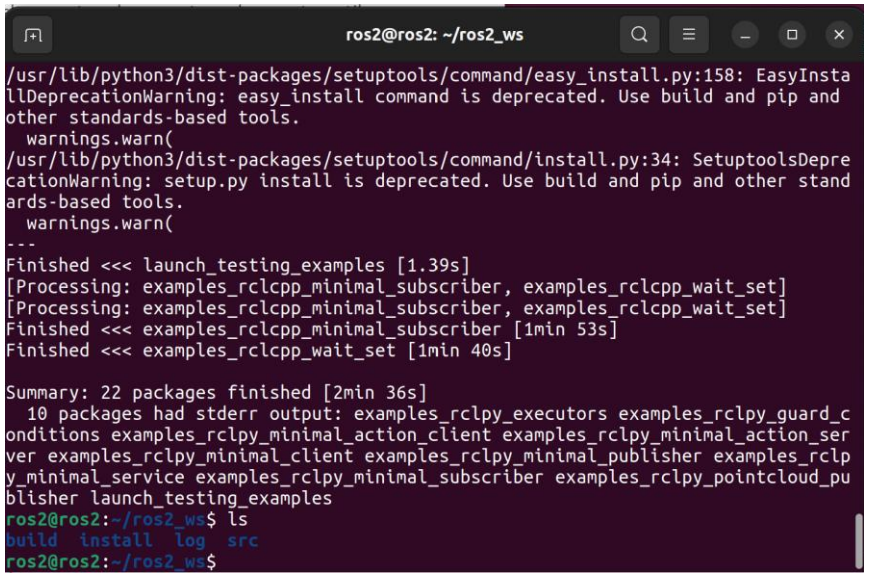
colcon build

- Kako stvaranje (*engl. build*) poput *amnet_cmake* ne podržava koncept *devel* direktorija, potrebno je da su paketi instalirani
- *colcon* podržava opciju *--symlink-install* koja omogućuje promjenu u instaliranim datotekama promjenom datoteka u *source* direktoriju (npr. *Python* programi ili drugi koji se ne moraju kompajlirati)
- Prije pokretanja *colcon build* naredbe, potrebno je nalaziti se u radnom direktoriju (*ros2_ws*)

```
cd
```

```
cd ~/ros2_ws
```

```
colcon build --symlink-install
```



source radnog direktorija

- Nakon što je *colcon build* završio unutar radnog direktorija *ros2_ws* dodane su još tri nova pod-direktorija
- Moguće je testirati radni direktorij nakon *colcon build*-a

```
colcon test
```

- Prije pokretanja primjera, potrebno je *source*-at novo napravljeni nadređeni (*engl. overlay*) direktorij
- Unutar *ros2_ws* pokrenuti *setup.bash* koji se nalazi u *install* pod-direktoriju

```
source install/setup.bash
```

colcon build

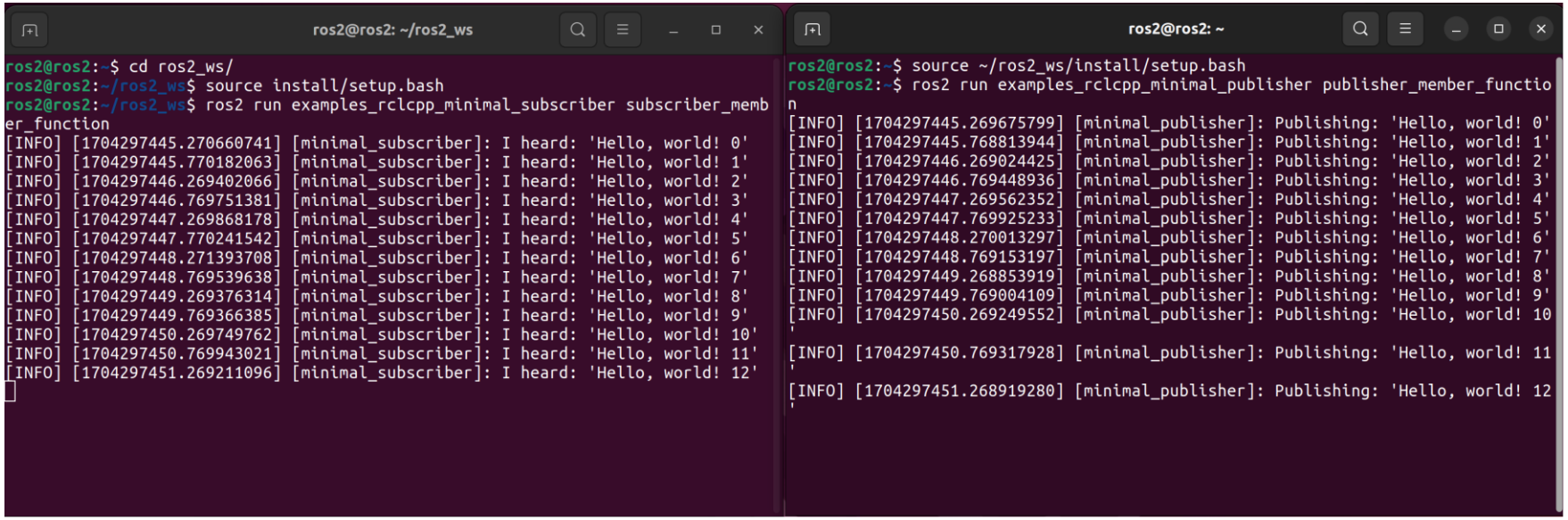
- U *source*-anom terminalu pokrenuti *subscriber* čvor

```
ros2 run examples_rclcpp_minimal_subscriber subscriber_member_function
```

- Otvoriti novi terminal, *source*-ati radni direktorij i pokrenuti *publish*-er čvor

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 run examples_rclcpp_minimal_publisher publisher_member_function
```



Novi radni direktorij

- Potrebno je napraviti novi radni direktorij *vjezba_ws* i pod-direktorij *src*

```
mkdir –p ~/vjezba_ws/src
```

- Preuzeti primjer preko *git*-a

```
cd ~/vjezba_ws/src
```

```
git clone https://github.com/ros/ros_tutorials.git -b humble
```

- *ros_tutorials* sadrži *turtlesim* paket, ostali paket sadrže datoteku *COLCON_IGNORE* te neće biti *build*-ani

Instalacija ovisnosti

- Prije *build*-anja preuzetog paketa, potrebno je instalirati sve ovisnosti (*engl. dependencies*) kako bi se preuzeti paket uspješno *build*-ao
- Potrebno je preuzeti i inicijalizirati *rosdep* paket koji omogućuje instalaciju svih osvisnosti

```
sudo apt install python3-rosdep2
```

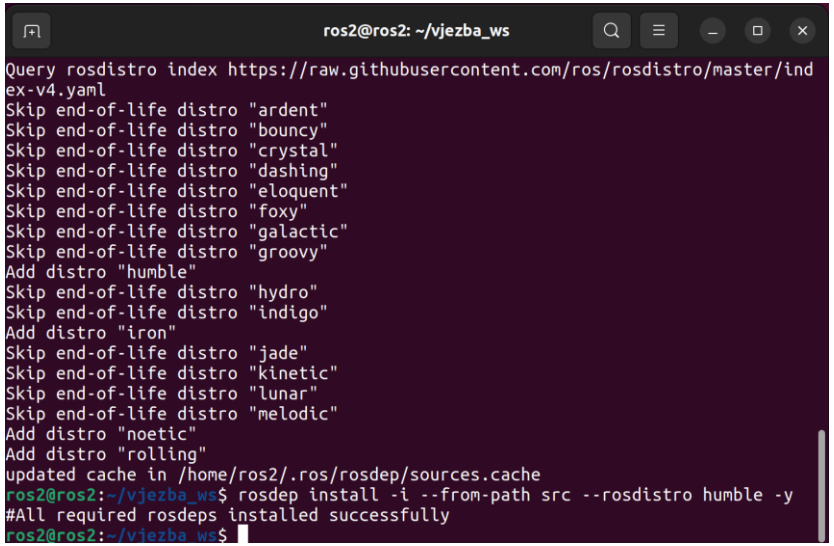
```
rosdep update
```

- Potom je potrebno otići u direktorij *vjezba_ws* i pokrenuti sljedeće

```
cd ..
```

```
rosdep install -i --from-path src --rosdistro humble -y
```

- Svaki paket sadrži informacije o svim ovisnostima u *package.xml*



colcon build

- Potrebno je *build*-at paket

```
colcon build
```

- Korisni argumenti *colcon build* naredbe:
  - *--packages-up-to* – *build*-a pakete koje želimo i njihove ovisnosti, ali ne cijeli radni direktorij
  - *--symlink-instal* – mijenja datoteke u *src*-u, dovoljno kad mijenjamo *Python* skripte koje ne zahtijevaju *rebuild*-anje
  - *--event-handlers console_direct+* - prikazuje izlaz prilikom *build*-anja, inače se mogu pronaći u *log* direktoriju

source

- Potrebno je *source*-ati novo napravljeni radni direktorij

```
source install/local_setup.bash
```

- *local_setup.bash* dodaje pakete dostupne samo nadređenom direktoriju

VAŽNO

```
source /opt/ros/humble/setup.bash
```

```
source ~/radni_direktorij/install/local_setup.bash
```

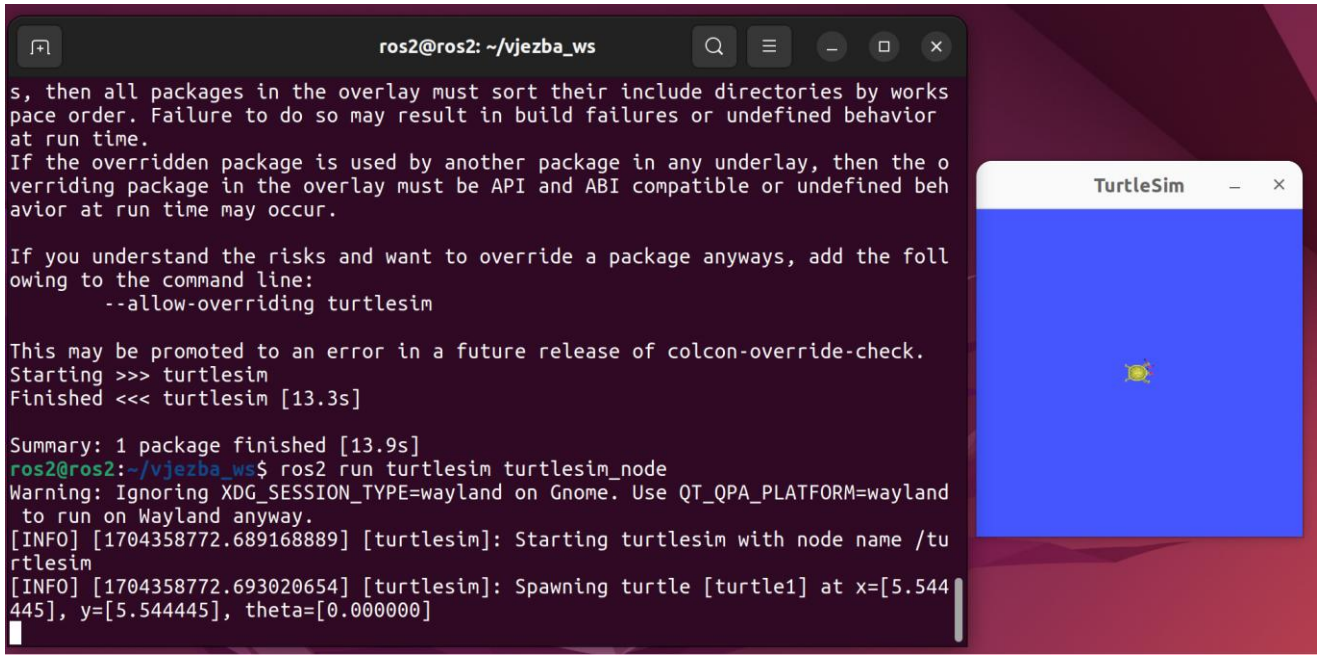
ISTO KAO

```
source ~/radni_direktorij/install/setup.bash
```

Pokretanje *turtlesim_node*

```
ros2 run turtlesim turtlesim_node
```

- Kako znati koji *turtlesim_node* se pokrenuo, podređeni (*engl. underlay*) ili nadređeni (engl. *overlay)*
  - Moguće je modificirati i *rebuild*-ati pakete u nadređenom radnom direktoriju odvojeno od podređenog glavnog
  - Nadređeni direktorij ima prednost u odnosu na podređeni



Modificiranje *turtlesim* paketa

- Kako bi pokazali da ROS uzima *ovarlay*, modificirat ćemo dio paketa *turtlesim*
- Potrebno je otići u:

```
cd ~/vjezba_ws/src/ros_tutorials/turtlesim/src
```

- I otvoriti *turtle_frame.cpp*

```
code turtle_frame.cpp
```

- I urediti 52 liniju koda iz *„TurtleSim”* u *„Moja kornjaca”* te spremiti s *Ctrl + s*

```
47 , path_painter_(&path_image_)
48 , frame_count_(0)
49 , id_counter_(0)
50 {
51   setFixedSize(500, 500);
52   setWindowTitle("Moja kornjaca");
53
54   srand(time(NULL));
```

Modificiranje *turtlesim* paketa

- Potrebno je vratiti se u *vjezba_ws* i *build*-ati radni direktorij

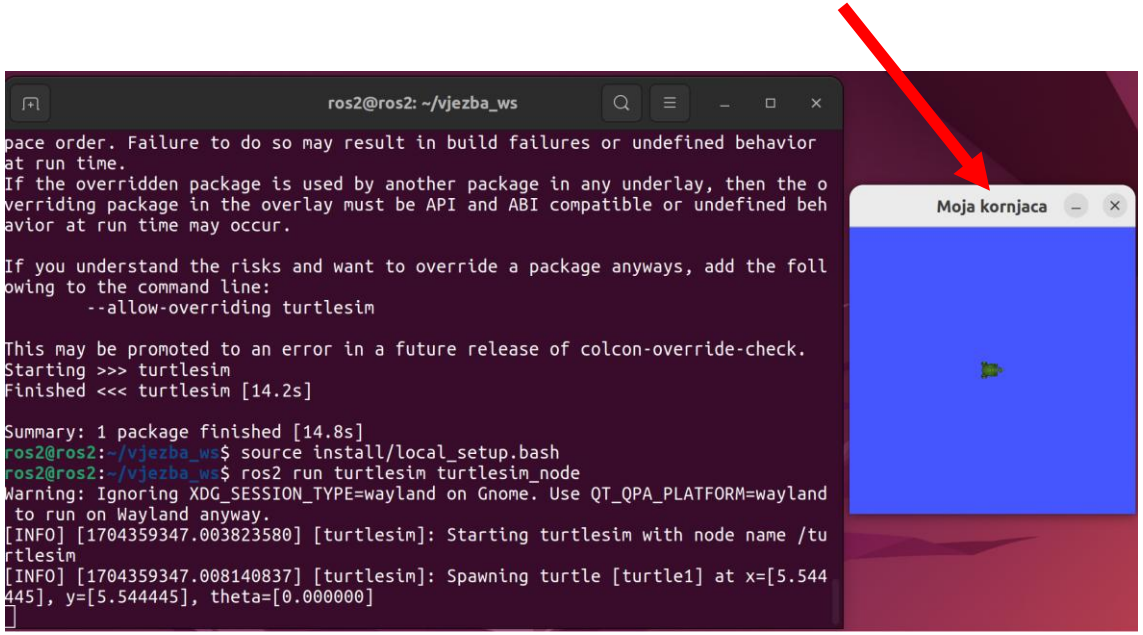
```
cd ~/vjezba_ws/
```

```
colcon build
```

- *Source*-ati radni direktorij i pokrenuti *turtlesim_node*

```
source install/local_setup.bash
```

```
ros2 run turtlesim turtlesim_node
```



Stvaranje paketa

- Paketi služe za organizaciju programa u ROS okruženju
- Paketi koriste *amnet build* sistem i *colcon build* alat za stvaranje prorgama
- Izrada paketa se može se ostvariti preko *CMake*-a ili *Python*-a
- Koristimo *Python*, za više o korištenju *CMake*-a, pogledati službene stranice ROS-a
- https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html

Primjer paketa

- Najjednostavniji paket *my_package* napravljen za *Python* izgleda ovako:

```
my_package/
    package.xml
    resource/my_package
    setup.cfg
    setup.py
    my_package/
```

- *package.xml* – sadrži glavne informacije o paketu
- *resource/my_package* – marker datoteka za paket
- *setup.cfg* – datoteka potrebna kada paket programe koji se mogu pokrenuti (*engl. executables)* tako da ih *ros2 run* može pronaći
- *setup.py* – sadrži instrukcije kako se instalira paket
- *<package_name>* - direktorij s istim imenom kao paket kojeg koriste ROS 2 alati za pronalazak paketa, sadrži *__init__.py*

Primjer paketa

- Radni direktorij može imati paketa koliko god želimo, svaki u svom direktoriju
- Moguće je imati pakete s različitim načinima *build*-anja (*Python, CMake* itd.)
- Nije moguće imati ugniježdene pakete
- Najbolje je imati sve pakete u *src* datoteci, kako bi najviša razina radnog direktorija bila čista

```
workspace_folder/
    src/
      cpp_package_1/
          CMakeLists.txt
          include/cpp_package_1/
          package.xml
          src/

      py_package_1/
          package.xml
          resource/py_package_1
          setup.cfg
          setup.py
          py_package_1/
```

Izrada paketa

- U radnom direktoriju *vjezba_ws* ćemo napraviti paket

```
cd ~/vjezba_ws/src
```

- Za izradu paketa koristi se sljedeća naredba

```
ros2 pkg create --build-type ament_python --license Apache-2.0 <package_name>
```

- Mi ćemo koristiti argument *--node-name* koji odmah napravi *Hello World* program

```
ros2 pkg create --build-type ament_python --license Apache-2.0 --node-name my_node
my_package --license Apache-2.0
```

Izrada paketa

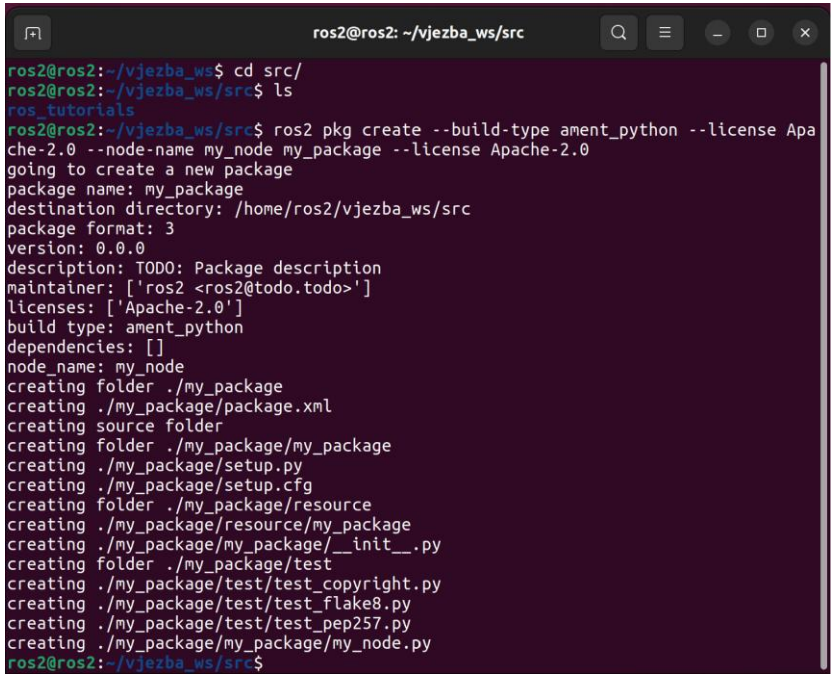
- Pokretanjem prethodne naredbe, automatski se generiraju datoteke potrebne za paket
- Potom je potrebno *build*-ati radni direktorij

```
cd ..
```

```
colcon build
```

- Ako u radnom direktoriju imamo više paketa, može se koristiti

```
colcon build --packages-select my_package
```



Korištenje paketa

- Prije pokretanja čvorova unutar paketa, potrebno je *source*-ati radni direktorij

```
source ~/vjezba_ws/install/local_setup.bash
```

- Pokrenuti čvor

```
ros2 run my_package my_node
```

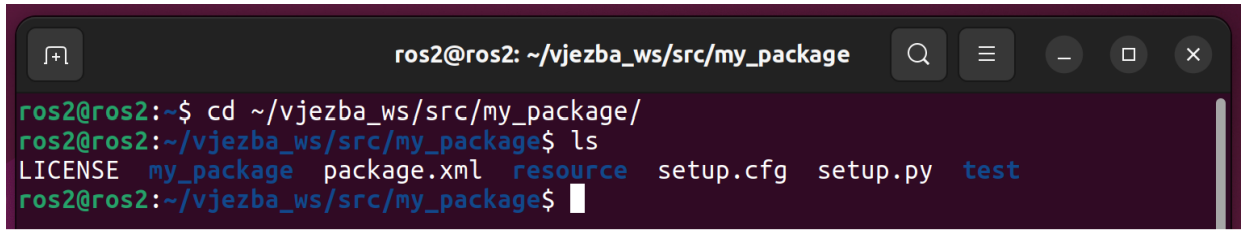
```
  1 package had stderr output: my_package
ros2@ros2:~/vjezba_ws$ source install/local_setup.bash
ros2@ros2:~/vjezba_ws$ ros2 run my_package my_node
Hi from my_package.
ros2@ros2:~/vjezba_ws$
```

Sadržaj paketa

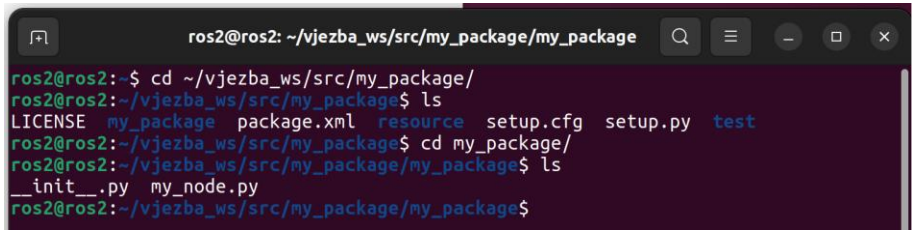
- Unutar direktorija *vjezba_ws/src/my_package* vidjet će se automatski generirane datoteke i mape

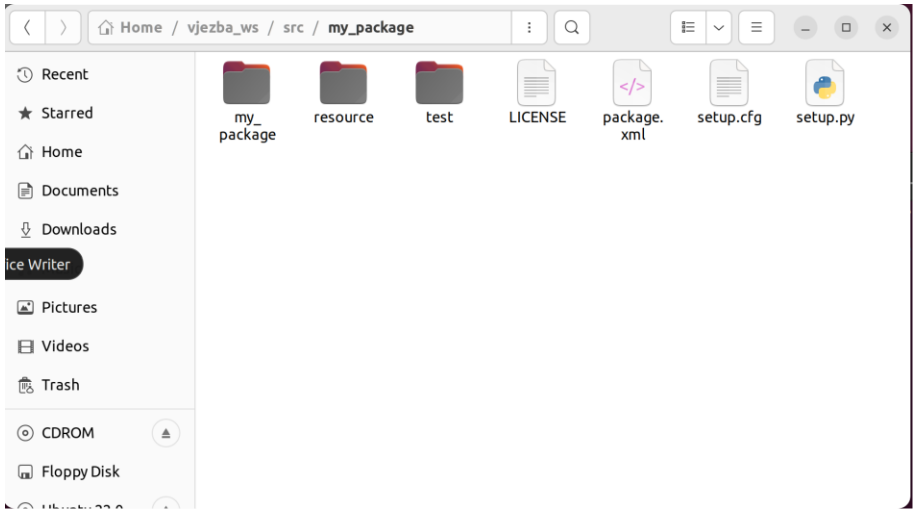
```
cd ~/vjezba_ws/src/my_package
```

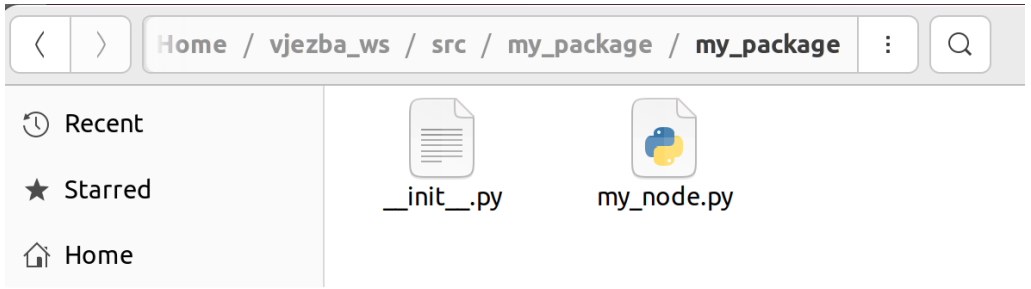
```
ls
```



Čvor *my_node* se nalazi u direktoriju *my_package*







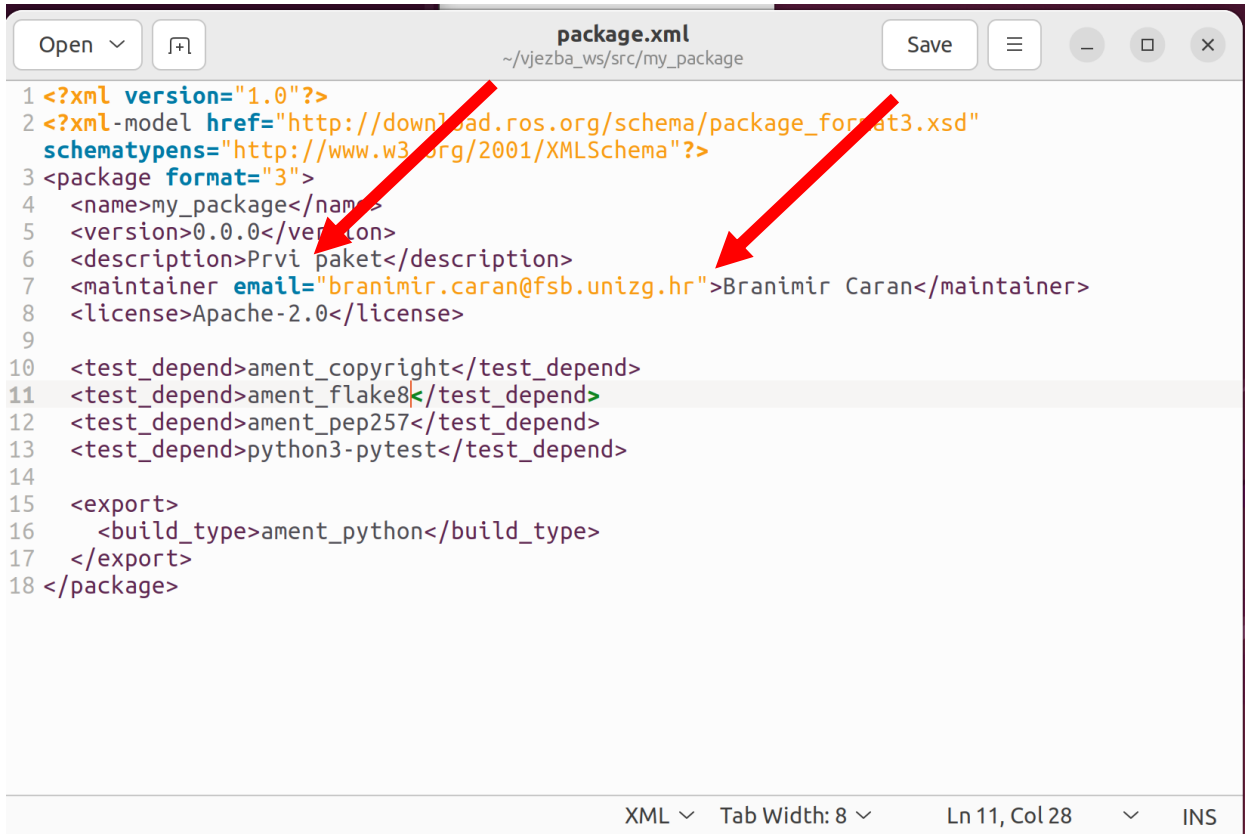
Uređivanje *package.xml* datoteke

- Potrebno je otići u paket *my_package* i urediti datoteku *package.xml*

```
cd ~/vjezba_ws/src/my_package
```

```
code package.xml
```

- Dijelovi s *_depend* sadrže ovisnosti o drugim paketima
- *my_package* je jednostavan paket koji nema nikakvih ovisnosti



Uređivanje *setup.py* datoteke

- Unutar *setup.py* potrebno je također urediti polja kao i u *package.xml* datoteci da budu ista

```
cd ~/vjezba_ws/src/my_package
```

```
code setup.py
```

setup.py
~/vjezba_ws/src/my_package

```python
from setuptools import find_packages, setup

package_name = 'my_package'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Branimir Caran',
    maintainer_email='branimir.caran@fsb.unizg.hr',
    description='Prvi paket',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'my_node = my_package.my_node:main'
        ],
    },
)
```

Python 2 Tab Width: 8 Ln 16, Col 31 INS

Kreiranje vlastitog *publisher*-a

- U radnom direktoriju *vjezba_ws* potrebno je napraviti paket *py_pubsub*

```
cd ~/vjezba_ws/src/
```

```
ros2 pkg create --build-type ament_python --license Apache-2.0 py_pubsub
```

- Potom otići u pod-direktorij *py_pubsub* i preuzeti primjer publisher-a

```
cd py_pubsub/py_pubsub
```

```
wget
https://raw.githubusercontent.com/ros2/examples/humble/rclpy/topics/minimal_publisher/examples_rclpy_minimal_publisher/publisher_member_function.py
```

- Otvoriti preuzeti čvor

```
code publisher_member_function.py
```

```python
# Namjerno izbrisan komentar
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MinimalPublisher(Node):

    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: %d' % self.i
        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1

def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_publisher.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Uređivanje *package.xml* i *setup.py*

- Potrebno je urediti *package.xml* i dodati ovisnosti

```
cd ~/vjezba_ws/src/py_pubsub
```

```
code package.xml
```

```
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend
>
```

Open package.xml ~/vjezba_ws/src/py_pubsub Save

```xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>py_pubsub</name>
  <version>0.0.0</version>
  <description>Minimalni publisher i subscriber</description>
  <maintainer email="branimir.caran@fsb.unizg.hr">Branimir Caran</maintainer>
  <license>Apache-2.0</license>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

XML Tab Width: 8 Ln 13, Col 44 INS

Uređivanje *package.xml* i *setup.py*

- Potrebno je urediti *setup.py*

```
cd ~/vjezba_ws/src/py_pubsub
```

```
code setup.py
```

```
'talker =
py_pubsub.publisher_member_function:main',
```

Open | *setup.py ~/vjezba_ws/src/py_pubsub | Save

```python
from setuptools import find_packages, setup

package_name = 'py_pubsub'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Branimir Caran',
    maintainer_email='branimir.caran@fsb.unizg.hr',
    description='Minimalni publisher i subscriber',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
                'talker = py_pubsub.publisher_member_function:main',
        ],
    },
)
```

Python 2 | Tab Width: 8 | Ln 23, Col 69 | INS

Kreiranje vlastitog *subsciber*-a

- Potrebno je otići u pod-direktorij *py_pubsub* i preuzeti *subscriber*-a

```
cd ~/vjezba_ws/src/py_pubsub/py_pubsub
```

```
wget
https://raw.githubusercontent.com/ros2/examples/humble/rclpy/topics/minimal_subscriber/examples_rclpy_minimal_subscriber/subscriber_member_function.py
```

- Otvoriti preuzeti čvor

```
code subscriber_member_function.py
```

```python
#Namjerno izbrisan komentar
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10)
        self.subscription  # prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)

def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    rclpy.spin(minimal_subscriber)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_subscriber.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Uređivanje *package.xml* i *setup.py*

- *Package.xml* datoteka ostaje ista, a *setup.py* je potrebno promijeniti

```
cd ~/vjezba_ws/src/py_pubsub
```

```
gedit setup.py
```

```
'listener =
py_pubsub.subscriber_member_function:main',
```

setup.py
~/vjezba_ws/src/py_pubsub

```python
from setuptools import find_packages, setup

package_name = 'py_pubsub'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Branimir Caran',
    maintainer_email='branimir.caran@fsb.unizg.hr',
    description='Minimalni publisher i subscriber',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
                'talker = py_pubsub.publisher_member_function:main',
                'listener = py_pubsub.subscriber_member_function:main',
        ],
    },
)
```

Python 2 | Tab Width: 8 | Ln 12, Col 29 | INS

build i pokretanje

- Potrebno je otići u radni direktoric i pokrenuti *rosdep install*

```
cd ~/vjezba_ws/
```

```
rosdep install -i --from-path src --rosdistro humble -y
```

- Potom je potrebno *build*-at i *source*-at radni direktorij

```
colcon build --packages-select py_pubsub
```

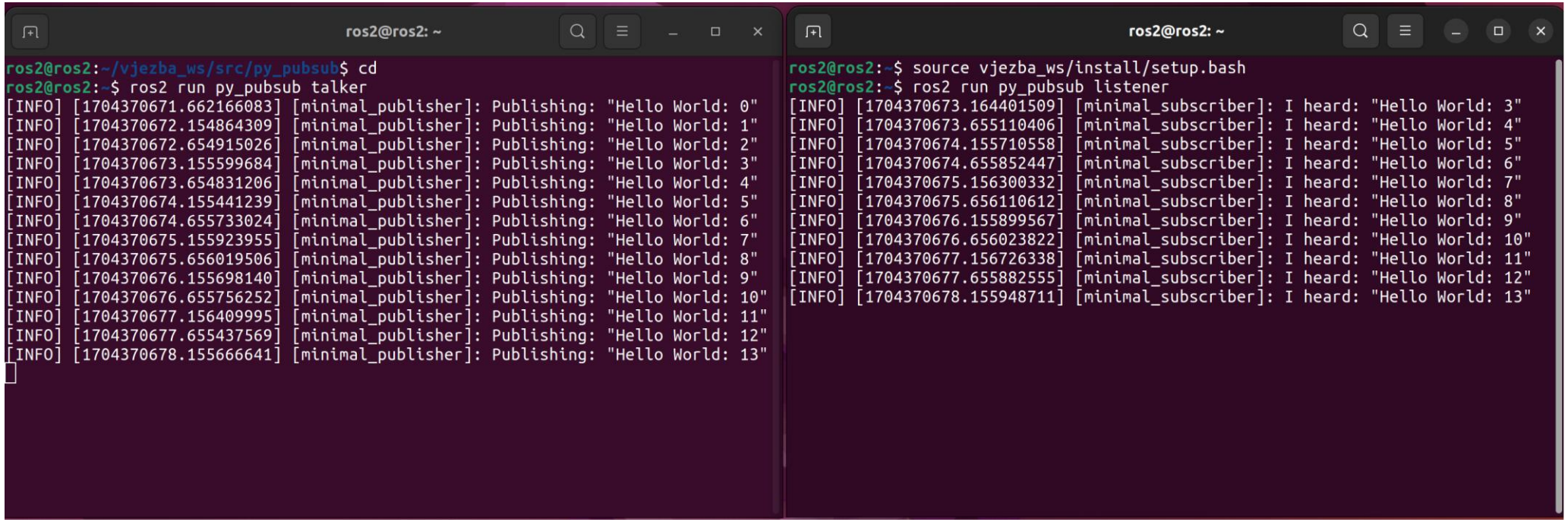
```
source install/setup.bash
```

- Pokretanje *talker* čvora

```
ros2 run py_pubsub talker
```

build i pokretanje

- Potrebno je *source*-at radni direktorij i pokrenuti čvor *listener* u novom terminalu

```
source ~/vjezba_ws/install/setup.bash
```

```
ros2 run py_pubsub listener
```